# 软件需求分析与设计

## 第六章框架梳理

仅供学习参考

# 目 录


**一、 分布式物理体系结构 - 1 -**

（一） 体系结构 - 1 -

1. 体系结构的设计目标 - 1 -

2. 体系结构设计的分类 - 1 -

（二） 对等体系结构（P2P） - 1 -

1. P2P 的中心组织原则 - 1 -

2. P2P 的优点 - 1 -

3. P2P 的缺点 - 1 -

4. P2P 的用途 - 1 -

（三） 分层体系结构 - 1 -

1. 层次的组成 - 1 -

2. 应用服务与业务服务的区别 - 1 -

3. 分层体系结构的分类 - 2 -

（四） 数据库为中心的体系结构 - 2 -

1. 定义和性质 - 2 -

**二、 多层逻辑体系结构 - 3 -**

（一） 体系结构的复杂性 - 3 -

1. 复杂性的分类 - 3 -

2. 空间认知复杂性 - 3 -

3. 网络结构复杂性 - 3 -

4. 层次的结构复杂性 - 4 -

（二） 体系结构模式 - 4 -

1. 外观模式（Facade） - 4 -

2. 抽象工厂模式（Abstract Factory） - 4 -

3. 责任链模式（Chain of responsibility） - 5 -

4. 观察者模式 / 出版—订阅模式（Observer / Publish—Subscribe） - 5 -

5. 中介者模式（Mediator） - 5 -

**三、 体系结构建模 - 7 -**

（一） 包（package） - 7 -


仅供学习参考

1. 包的定义 - 7 -
2. 包的性质 - 7 -
（二） 构件（component） - 7 -
1. 构件的定义 - 7 -
2. 构件的特性 - 7 -
3. 构件 vs 包 - 7 -
4. 构件 vs 类和接口 - 8 -
（三） 结点（node） - 8 -
1. 结点的定义 - 8 -
**四、 程序设计与复用原则 - 9 -**
（一） 系统设计的组成 - 9 -
1. 架构设计——框架（framework） - 9 -
2. 详细设计——前端（界面 GUI），后端（数据库） - 9 -
3. 程序设计——对界面设计和数据库设计的扩展 - 9 -
（二） 类的内聚与耦合 - 9 -
1. 类的内聚（cohesion） - 9 -
2. 类的耦合（coupling） - 9 -
3. 复用策略 - 10 -
**五、 协作建模 - 11 -**
（一） 协作 - 11 -
1. 协作的定义 - 11 -
2. 协作的表示 - 11 -
（二） 复合结构（Composite） - 11 -
1. 复合结构的定义 - 11 -
2. 复合结构的表示 - 11 -

仅供学习参考

# 一、分布式物理体系结构

## （一）体系结构

1. 体系结构的设计目标

   (1) 主要目标都是**降低耦合**

2. 体系结构设计的分类

   (1) 物理体系结构

   ① 解决 **C/S** 以及 C/S 之间所需要的任何**中间件**的问题

   ② 使用结点和**部署图**

   (2) 逻辑体系结构

   ① 客户机（client）：发起请求，请求服务器的计算进程（同一时间只能访问一台服务器）

   ② 服务器（server）：接收并处理客户机发来的请求，服务于客户请求的计算进程

   ③ 对 C/S 体系结构进行扩展就能表示任意的分布式系统

## （二）对等体系结构（P2P）

1. P2P 的中心组织原则

   (1) 任何同位体都可以**自由地**直接与任何其他同位体通信，而不需要使用中间服务器

   (2) 系统中任何节点可能既是客户机，也是服务器

2. P2P 的优点

   (1) 没有管理节点，使网络附在得到均衡

   (2) 对于网络故障和单个同位体故障，P2P 更易于快速恢复，且易于扩展和适应

3. P2P 的缺点

   (1) 当系统总吞吐量达到最大时，就要考虑**网络流量最小化**问题

   (2) 进程之间可能存在**死锁**问题

   (3) 需要确保系统具有一致的**反应时间**

4. P2P 的用途

   (1) 主要用在分布式的文件共享系统（六维空间）

## （三）分层体系结构

1. 层次的组成

   (1) 层次结构中间的每一层既是客户机也是服务器

   (2) 在硬件层与软件层之间采用**层次依赖方法**

   (3) 典型的三层体系结构：GUI 客户机、单独的业务逻辑、数据库服务器

   (4) 只允许**相邻层**间进行通信，将复杂性从指数级将为**多项式级**

2. 应用服务与业务服务的区别

仅供学习参考

(1) 应用服务：是每一个运行在数据库上的**单独**程序所做的

(2) 业务服务：是数据库**强迫**（通过编程实现）所有运行程序都要遵循的业务规则

3. 分层体系结构的分类

(1) 胖客户体系结构（a client on steroids）——应用逻辑被编译到客户机

(2) 瘦客户体系结构（a skinny client）——应用逻辑被编译到服务器

(3) 中介者体系结构——一部分应用逻辑被编译到客户端，一部分被编译到服务器

（四）数据库为中心的体系结构

1. 定义和性质

(1) 数据库程序被称为**存储过程**，可以从客户端程序通过一般的过程/函数调用它

(2) **触发器**作为一种特殊的存储过程，不能被显示地调用

(3) 当试图更改数据库内容时，触发器自动触发是一种经常应用于属性或关联的约束

仅供学习参考

# 二、多层逻辑体系结构

## （一）体系结构的复杂性

### 1. 复杂性的分类

(1) 问题复杂性

① 由问题域本身复杂性定义，也称计算复杂性

(2) 算法复杂性

① 目标：度量软件算法的效率（时间复杂度、空间复杂度）

(3) 结构复杂性

① 定义：类或结构之间关联关系的复杂性

② 目标：建立软件结构之间的关系及易于维护和演化，即降低耦合度的问题

③ 结构复杂性的度量强调程序对象之间的依赖（没有供应者，客户的语义是不完整的）

1) 如果系统中的所有依赖都被标识和理解，则说系统有适应能力（可理解、可维护、可伸缩）

2) 可以根据不同粒度对象（构件、包、类、方法）来识别依赖

④ 度量可以增强可维护性和可伸缩性质量

(4) 认知复杂性

① 度量理解软件做出的努力，即捕获程序的逻辑流，理解软件所需要的工作量

② 度量可以增强可理解性质量

### 2. 空间认知复杂性

(1) 目标：度量软件工程师能看懂代码（build a mental model）时目光所移动的代码的行数

(2) 计算公式：

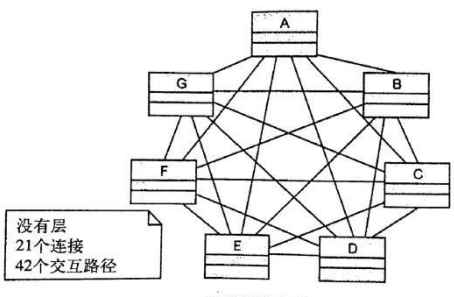
① 函数空间复杂性：$FC=\sum_{i=1}^{total_calls} dist_i$（*dist* 指从函数调用到函数定义的代码行数）

② 程序空间复杂性：$PC=\sum_{i=1}^{total_functions} FC_i$（*total_functions* 是程序中函数的数量）

(3) 弱点：以代码行计算距离，适用于面向过程编程，但不适用于面向对象编程

### 3. 网络结构复杂性

(1) 计算公式：累积的类依赖：$_{net}CCD=n(n-1)$（n 是结点的数量）




网络的复杂性

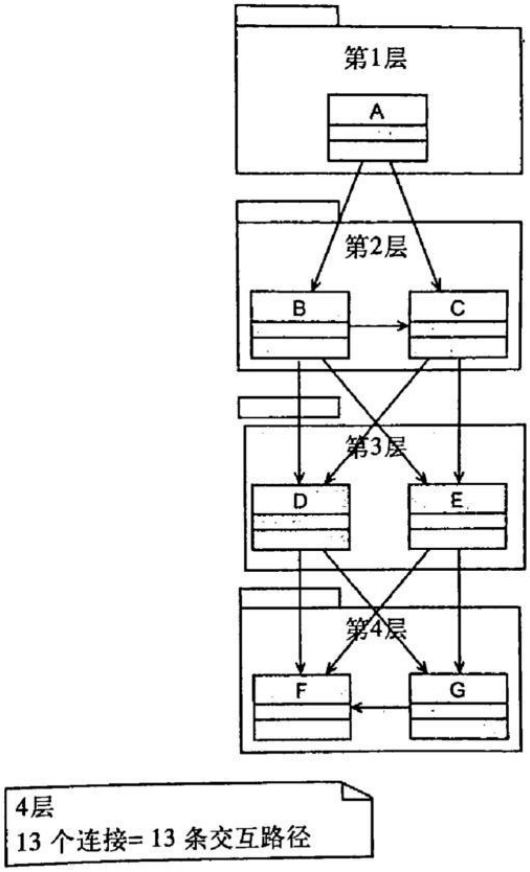
4. 层次的结构复杂性

(1) 计算公式：$_{holarchy}CCD=\sum_{i=1}^{n}\frac{size(l_i)\times(size(l_i)-1)}{2}+\sum_{i=1}^{n}\sum_{j=1}^{l_i}(size(l_i)\times size(p_j(l_i)))$

① 式中第 1 部分计算每一层内所有类之间的潜在单向路径的数量

② 式中第 2 部分计算每一对相邻层的类之间的潜在单向路径的数量

其中：$size(l_i)$为第 $l_i$ 层的对象数；$l_i$ 为第 $l_i$ 层的双亲数；$p_j(l_i)$为第 $l_i$ 层的第 $j$ 个双亲




层次结构的复杂性

（二）体系结构模式

1. 外观模式（Facade）

(1) 定义：使子系统更易于使用的高层接口，高层接口可以是具体类（域类）或抽象类

(2) 目标：减少子系统之间的通信和依赖性（耦合）

(3) 副作用：层中的类隐藏在外观对象的后面

(4) 适用场景

① 子系统越来越复杂，增加外观模式是供简单调用接口

② 构建多层系统结构，利用外观对象作为每层的入口，简化层间调用

2. 抽象工厂模式（Abstract Factory）

(1) 定义：提供一个接口，创建相关的或依赖的对象，而不需要说明它们的具体类

(2) 目标：通过访问隐藏在抽象工厂接口后面的几个对象家族之一，使得应用的表现行为不同

(3) 优点

① 有利于增加系统应用的整体灵活性，可以在类的内部对产品族进行约束。

(4) 缺点

① 假如产品族中需要增加一个新的产品，则几乎所有的工厂类都需要进行修改

(5) 适用场景

① 当应用需要根据运行时环境使用不同资源时，启用不同的 GUI 窗口或以不同的语言显示信息

② 一个继承体系中存在多个抽象类，并且各个抽象类中的实现类之间有一定的关联或者约束

3. 责任链模式（Chain of responsibility）

(1) 定义：委托概念的变体，是消息的引用链（类似于递归）

(2) 目标：通过将处理问题的机会给多个对象来避免问题的发送者和接收者之间的耦合

(3) 实现模式

① 双亲—子模型（parent—child model）：若子对象没有处理能力，就返回给其双亲，直到解决

② 容器——被包含模型（container—contained model）

(4) 适用场景

① 多种面向对象的 GUI 活动，如 GUI 帮助功能、构建布局、格式编排及定位

② 可能处于不同的体系结构层的类集合要对特定的初始消息做出响应

③ 实现 PCBMER 框架中的 NCP（相邻通信原则）

4. 观察者模式 / 出版—订阅模式（Observer / Publish—Subscribe）

(1) 定义：一般是一种一对多的依赖关系，可以有任意个观察者对象同时监听某一个对象

(2) 目标：当一个对象变更状态时，其所有依赖者都被通知到并自动更新

(3) 优点

① Subject 和 Observer 之间是松偶合的，分别可以各自独立改变

② 发送广播通知时，无须指定具体的 Observer，Observer 可以自己决定是否要订阅 Subject 的通知

(4) 缺点

① 观察者只知道被观察发生变化，而无法知道是如何发生变化的

② 如果有很多个观察者，一个个通知比较耗时

(5) 适用场景

① 对一个对象状态的更新，需要其他对象同步更新，而且其他对象的数量动态可变

② 对象仅需要将自己的更新通知给其他对象，而不需要知道其他对象的细节

5. 中介者模式（Mediator）

(1) 定义：用一个中介者对象来封装一系列的对象交互，使得各对象不需要显式地相互引用，从而使其松散耦合，而且可以独立地改变它们之间的交互

(2) 目标：将复杂的处理规则（包括复杂的业务规则）组织到专门的中介者类中

仅供学习参考

(3) 优点

① 将原有的一对多的依赖变成了一对一的依赖，降低类间的耦合

(4) 缺点

① 中介者会变得庞大且复杂，原本多个对象直接的相互依赖变成了中介者和多个同事类的依赖关系，同事类越多，中介者的逻辑就越复杂

(5) 适用场景

① 当对象之间的交互变多时，为了防止一个类会涉及修改其他类的行为，可以使用中介者模式

仅供学习参考

# 三、体系结构建模

## （一）包（package）

1. 包的定义

(1) 用于表示一组类（或其他建模元素，如用例），使高度相关的类聚合

(2) 包用文件夹图标来表示

2. 包的性质

(1) 包可以嵌套，外层包可以直接访问包括在它的嵌套包中的任何类

(2) 一个类只能在一个包中定义，但是可以通过引用出现在其他包中

(3) 包之间的关系：泛化和依赖

① 包之间的泛化也意味着依赖

② 依赖使从子类包到超类包，超类包中的变更会影响到子类包

## （二）构件（component）

1. 构件的定义

(1) 是系统的物理部分、实现一个片段或一个软件程序（.exe）

2. 构件的特性

(1) 构件是独立的部署单元——即构件里面的部分不能再分了，不可以部署构件的一部分

(2) 构件是第三方组装单元——它是充分文档化和自包含，可以被第三方插入到其他构件中

(3) 构件没有持久状态——不能与它的拷贝区分开，在任何给定的应用系统中，一个特定的构件最多有一个拷贝

(4) 构件是系统的可替换部分——相同接口的构件可以互相替换

(5) 构件可以完成清晰的功能，并且是逻辑耦合和物理耦合的

(6) 构件可以嵌套在其他构件中

3. 构件 vs 包

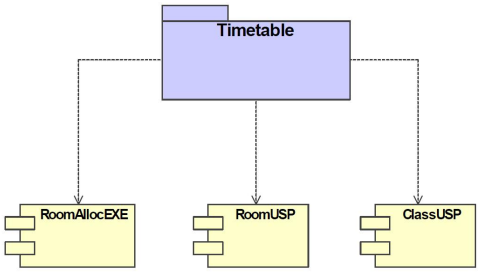
(1) 从逻辑上讲，每一个类属于一个单一的包

(2) 从物理上讲，每个类都至少由一个构件实现，并且一个构件可能只实现一个类。

(3) 抽象类和接口经常被多个构件实现

(4) 包是比构件更大的体系结构单元，倾向于以水平方式组织类——在应用域中静态接近的类

(5) 构件是对行为相近的类的垂直组织——类可能来自不同的域，但贡献于一个单一业务活动（如用例）




仅供学习参考

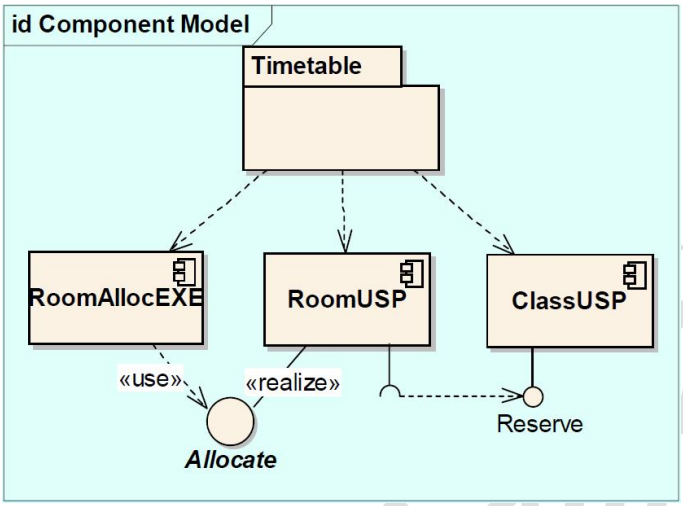
4. 构件 vs 类和接口

(1) 构件最终也来实现接口，构件只显示它所包含的类的某些接口

(2) 很多其他接口都被封装在构件中，只被协作的类在内部使用，对于其他构件是不可见的

(3) 构件是部署在某个计算机结点上的物理抽象

(4) 类表示逻辑事务，为了起到物理抽象的作用，不得不将其实现为构件




（三）结点（node）

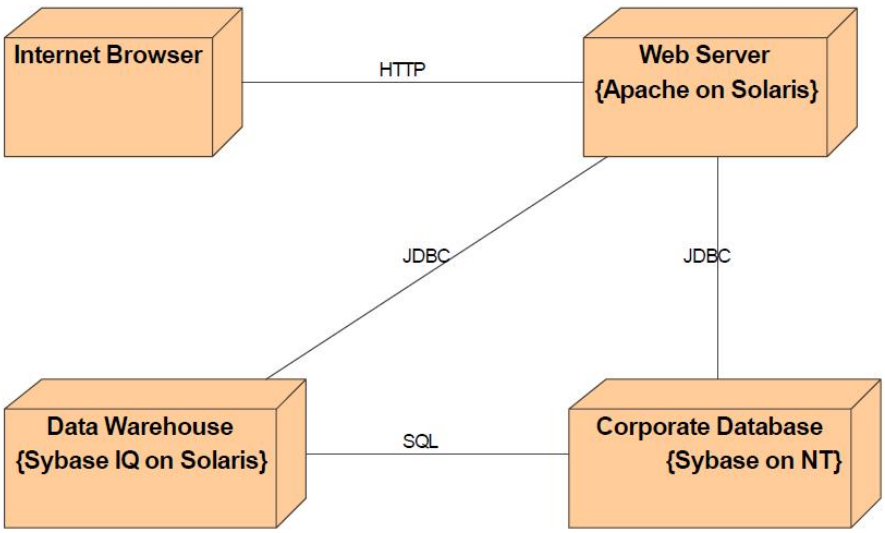
1. 结点的定义

(1) 结点——部署图（deployment diagram）中的计算资源（运行时的物理对象）

(2) 在 UML 中，结点用立方体表示

(3) 组件是最终要部署到某个结点上的，结点和其上的构件称为分布单元

(4) 结点是构件的一个载体——结点是硬件，构件是软件，结点运行构件



# 四、程序设计与复用原则

（一）系统设计的组成

1. 架构设计——框架（framework）
2. 详细设计——前端（界面 GUI），后端（数据库）
3. 程序设计——对界面设计和数据库设计的扩展

（二）类的内聚与耦合

1. 类的内聚（cohesion）
   (1) 类内聚是一个类内部**自确定**的程度
   (2) 它测量类**独立**的强度
   (3) 内聚（相对于耦合）越**强**越好
2. 类的耦合（coupling）
   (1) 类耦合是类之间**连接**的程度
   (2) 它测量类的相互**依赖性**
   (3) 耦合（相对于内聚）越**弱**越好
   (4) 耦合的形式
      ① X 包含 Y，或 X 具有指向 Y 的实例的属性，即 X 里的某个属性的类型就是 Y
      ② X 具有引用 Y 的实例的方法，如使用类型 Y 的参数或局部变量，或消息返回类型为 Y 的对象给 X
      ③ X 调用 Y 的服务（给 Y 发送消息）
      ④ X 是 Y 的直接子类或非直接子类
      ⑤ X 具有输入参数是类 Y 的方法
      ⑥ Y 是一个接口，X 实现了此接口
   (5) Demeter 法则（耦合规则）
      ① 类耦合应该尽量被限制在类的层次内，即层内耦合。层间耦合应该被最小化并小心引导
      ② 消息的目标只能是下面对象之一
         1) 方法的对象本身——C#和 Java 中的 this
         2) 方法型构中作为参数的一个对象
         3) 此对象的属性所引用的对象（包括属性集合中所引用的对象）
         4) 此方法创建的对象
         5) 全局变量引用的对象
      ③ 为了限制继承来的耦合，使用 Demeter 增强法则，将 3)规则限制在类本身定义的属性上
   (6) 存取方法（Accessor methods）和机械类（mindless classes）
      ① 存取方法定义观察者（get）和改变者（set）的操作

仅供学习参考

② 一个具有很多存取方法的类称为机械类，也称为无思维类，由其它类决定该类状态

(7) 动态分类（Dynamic classification）和混合实例内聚（mixed-instance cohesion）

① 动态分类——某个类一会儿属于 A 类，一会儿属于 B 类——编程中不支持

② 没有动态分类的支持造成了混合实例内聚，这是面向对象编程环境的**弱点**

③ 带有混合实例内聚的类中的某些对象可能是未定义的，这时候可用**泛化**消除这个问题

3. 复用策略

(1) 复用的**粒度**（Reuse Granularity）

① 类

② 构件

③ 解决方案

(2) 工具包复用

① 强调在**类**一级的代码复用

② 程序员通过调用某些类库中的具体类在程序中“填补空隙”

③ 程序的主体**不复用**——由程序员编写

④ 两种类型的工具包

1) 基础工具包（内置好的）——ArrayList，Set，String、Date 等

2) 体系结构工具包——作为系统软件的一部分，如操作系统、数据库软件（JDBC）

(3) 框架复用

① 框架强调**构件**级的设计复用

② 提供解决方案的基本框架，程序员完善框架——**可定制**的应用软件

③ 常用框架：ERP（企业资源规划系统），SAP，PeopleSoft，Baan，J.D.Edwards，J2EE，.NET

(4) 模式复用

① 模式强调在开发方法过程中的复用，解决某一问题的通用方法

仅供学习参考

# 五、协作建模

## （一）协作

### 1. 协作的定义

(1) 描述相互协作的角色（元素）结构，每个元素执行特定的功能，这些元素共同完成某项期望的功能

### 2. 协作的表示

(1) 协作被形象地表示为一个带有协作名字的**虚椭圆**

(2) **矩形图标**表示协作的实体（角色）

(3) 角色由**连接器**（connector）连接

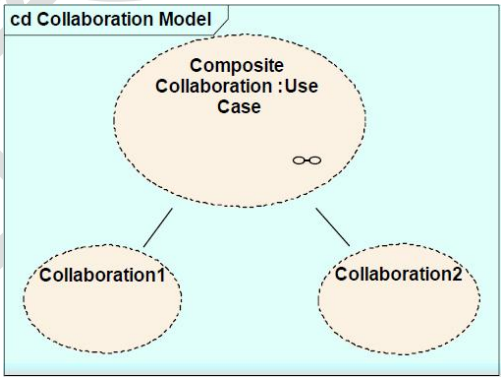
(4) 在协作名字的后面可以写上分类器名。

## （二）复合结构（Composite）

### 1. 复合结构的定义

(1) **复合结构**是指相互连接的元素组合，表示运行时的实例协作，通过通信连接取得某些共同的目标

(2) 本质上是一个**类图**，与交互图并行使用




### 2. 复合结构的表示

(1) **复合协作**表示法——含复合结构的**设计类图**

(2) 将所有的角色从椭圆中抽取出来放在**类**中（因此需要为角色定义任何缺失的类型）

(3) 将协作连接到类上，并为连接器定义上角色名（role1、role2）

(4) 由**下级协作**组成（实现），一个复合协作可以实现一个用例，而下级协作可以表示此用例的需求

